

Web Technologies

Assignment - II

Unit - IV

1. Illustrate Creating basic http server using Node.js.

Ans: Creating basic http server using Node.js

- Install the Node.js using following link:

<https://nodejs.org/en/download/prebuilt-installer>.

- We need to verify the Node.js installation as below:

- Node: Go to the Command Prompt and type `node -v` or `node -version`
- npm: Go to the Command prompt and type `npm -v` or `npm -version`.

- Step By Step Process to create a http server:

- First, ensure you have Node.js installed on your machine.
- Create a new directory for your project.
- Open the new directory using VS code editor
- Open Terminal and type the following command: `npm init -y`.
- Create a new file called `server.js`.

Code:

```
const http = require('http');
```

```
const hostname = '127.0.0.1';
```

```
const port = 3000;
```

```
const server = http.createServer((req, res) => {
```

```
  res.statusCode = 200;
```

```
  res.setHeader('Content-Type', 'text/plain');
```

```
  res.end('Hello, World! \n');
```

```
});
```

```
server.listen(port, hostname, () => {
```

```
  console.log('Server running at http://${hostname}:${port}/');
```

```
});
```

- Run: Run the application by typing `node server.js` at the terminal.

2. Explain http verbs along with simple code snippets for implementing them?

Ans: Http verbs along with simple codes. Snippets for implementing them.

~~GET~~

- GET: Retrieve data from a server. This method is idempotent, meaning multiple identical requests will yield the same result.

- POST: Send data to the server to create a new resource. This method is not idempotent, as repeated requests can create multiple resources.
- PUT: Update an existing resource or create it if it does not exist. This method is idempotent.
- PATCH: Partially update an existing resource. Unlike PUT, which replaces the entire resource, PATCH modifies only the specified fields.
- DELETE: Remove a resource from the server. This method is also idempotent; deleting the same resource multiple times has the same effect as deleting it once.
- HEAD: Similar to GET but only retrieves the headers, not the body. Useful for checking resource availability without transferring data.
- OPTIONS: Describe the communication options for the target resource, allowing clients to understand what methods are supported.
- TRACE: Echo back the received request, used mainly for diagnostic purposes.
- CONNECT: Establish a tunnel to the server identified by the target resource. Often used with HTTPS.

Code:

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/posts', (req, res) => {
  res.send('Retrieve all posts');
});

app.post('/posts', (req, res) => {
  res.send('Create a new post');
});

app.put('/posts/:id', (req, res) => {
  res.send('Update post with ID $ {req.params.id}');
});

app.delete('/posts/:id', (req, res) => {
  res.send('Delete post with ID $ {req.params.id}');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

3. Differentiate between GET and POST http methods:

Ans: GET: Retrieve data from a server. ~~This me~~

~~POST~~ POST: Send data to the server to create a new resource.

Difference between GET and POST HTTP methods

are:-

Feature	GET	POST
Purpose	Retrieve data	Submit / Create data.
Data location	URL (Query string)	Body of the request
Idempotency	Idempotent (Same output each time)	Non-idempotent (Can change state)
Use case	Fetching a blog post	Submitting a form or new post
Visibility	Data visible in URL	Data hidden from URL
Security	Less secure - data is visible in the URL and browser history	More secure - data is hidden from the URL, better for sensitive data (e.g., passwords)
Caching	Can be cached by browser	Not cached by default.
Usage Example:	viewing a user profile: / user?id=123	Submitting a registration form

Unit - 5

4. How / Explain how you render views in express application?

Ans: Render views in Express Application: To render views in express application we need to do the following:

- Install a View Engine [EJS]: EJS stands for Embedded JavaScript, is a simple templating language that allows developers to generate HTML markup with plain JavaScript.

npm install ejs

- Create a directory named views in the root of your project.

- Inside the views directory, create an index.ejs file: and write the following:

code
 <!-- views/index.ejs -->

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

in: <title><%= title %></title>

</head>

<body>

<h1><%= message %></h1>

</body>

</html>

- Set Up the View Engine in Express: Create a new file app.js in root directory with the following code:

```
code:
const express = require('express');
const app = express();
const port = 3000;
app.set('view engine', 'ejs');
app.set('views', './views');
app.get('/', (req, res) => {
  res.render('index', { title: 'Home', message: 'Hello World!' });
});
app.listen(port, () => {
  console.log('Server running at http://localhost:3000');
});
```

- Start the server and Test:
node app.js

5. Elaborate on session management in Express.js

Ans: Sessions in Express: Sessions in Express allow you to store user data across requests, which is useful for managing user authentication and other stateful interactions.

- Install express-session:

```
npm install express-session
```

- Configure Sessions in Express: Create a new file app.js in root directory and do the following:

```
code:
const express = require('express');
const session = require('express-session');
const app = express();
const port = 3000;
app.use(session({
  secret: 'your-secret-key',
  resave: false,
  saveUninitialized: false,
  cookie: { maxAge: 60000 }
}));
app.get('/', (req, res) => {
  res.send('Session example');
});
```


app.listen(port, () => {
 console.log('Server running at http://localhost:\${port}');
});

- Storing Data in the Session:

```
app.get('/login', (req, res) => {  
  req.session.user = { username: 'Vijay', role: 'admin' };  
  res.send('User logged in!');  
});
```

- Accessing Session Data:

```
app.get('/dashboard', (req, res) => {  
  if (req.session.user) {  
    res.send('Welcome, $ { req.session.user.username } !');  
  } else {  
    res.send('Please log in first.');
```

- Destroying Session Data:

```
app.get('/logout', (req, res) => {  
  req.session.destroy(err => {  
    if (err) return res.send('Error logging out');  
    res.send('You are logged out!');
```

Q. Explain File Upload concept in Express JS?

Ans: Handling file upload in Express.js is straight forward with the help of middleware like multer.

- Steps how to implement it:

- Create an sample app handling forms
- `npm install express`
- `npm install multer`: Multer is a middleware for handling multipart/form-data, primarily used for file uploads in express.
- Create a file `app.js`
- Write the following code:

- Code:

```
const express = require('express');  
const multer = require('multer');  
const path = require('path');  
const app = express();  
const port = 3000;  
const storage = multer.diskStorage({
```

```
destination: (req, file, cb) => {
```

```
  cb(null, 'uploads/');
```

```
},
```

```
filename: (req, file, cb) => {
```

```
  cb(null, Date.now() + path.extname(file, originalName));
```

```
},
```

```
};
```

```
const upload = multer({ storage: storage, limits: { fileSize: 5 * 1024 * 1024 } });
```

```
app.get('/', (req, res) => {
```

```
  res.send('
```

```
    <form action="/upload" method="POST" enctype="multipart/
```

```
form-data">
```

```
    <input type="file" name="file" />
```

```
    <button type="submit">Upload </button>
```

```
  </form>
```

```
  ');
```

```
});
```

```
app.post('/upload', upload.single('file'), (req, res) => {
```

```
  if (!req.file) return {
```

```
    return res.status(400).send('No file uploaded.');
```

```
  }
```

```
  res.send('File uploaded successfully: ${req.file.filename}');
```

```
});
```

```
app.listen(port, () => {
```

```
  console.log('Server running at http://localhost:${port}');
```

```
});
```

• Now run the code `node app.js`.

7. Write queries in MongoDB to insert, update and delete documents.

Ans: Insert Queries in MongoDB to insert, update and delete document are.

- Insert Document:

• Insert One:

```
db.wen.insertOne({
```

```
  name: "Vijay",
```

```
  city: "Hyd",
```

```
  department: "CSE"
```

```
});
```


• Insert many:

```
db.users.insertMany([
  { name: "Ajay", city: "Hyd", department: "CSIS" },
  { name: "Shiva", city: "Wgl", department: "ECE" }
]);
```

- Update Documents:

• Update One:

```
db.users.updateOne({ _id: ObjectId("668e55bb4714999ffe8e3e9") },
  { $set: { name: "Vijay Kumar", age: 35 } });
```

• Update Many:

```
db.users.updateMany({ name: "Shiva" },
  { $set: { name: "Shiva", age: 25 } });
```

- Delete Documents:

• Delete one:

```
db.users.deleteOne({ _id: ObjectId("668f62557442a9754ecb7f83") });
```

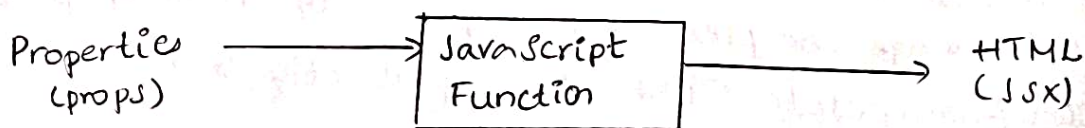
• Delete many:

```
db.users.deleteMany({ age: 32 });
```

Unit-V

8. Explain briefly function component and class component in React.

Ans: Function Components: These are simple JavaScript functions that accept props as an argument and return React element (JSX)



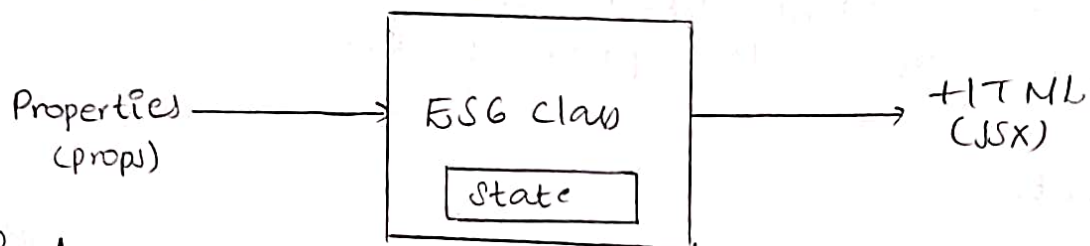
- features:

- Stateless by default but can manage state using React Hooks like `useState` or `useEffect`.
- Easier to write and test
- Lightweight and ~~pre~~ preferred for most use cases in modern React.

- Code:

```
function MyGreeting(props) {
  return <h1>Hello, {props.name}! </h1>;
}
export default MyGreeting.
```

Class Component: In react class component is a class that extends `React.Component` and gives access to `React.Component` class function.



- Features:

- Can manage state and have lifecycle methods (e.g., `ComponentDidMount`, `ComponentDidUpdate`).
- Heavier than functional components.
- Gradually being replaced by functional components with hooks.

- Code:

```
import React, {Component} from "react";
class MyGreeting extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}! </h1>;
  }
}
export default MyGreeting;
```

Q Explain any four hooks in React Js.

Ans: Hooks in React Js. are

~~- useState~~

- `useState()`: To manage state in functional component.
- `useEffect()`: To perform side effects (like API calls, DOM updates).
- `useContext()`: To access React context values.
- `useReducer()`: To manage complex state logic with reducer pattern.
- `useRef()`: To access/manipulate DOM elements directly or hold mutable values.
- `useMemo()`: To optimize performance by memoizing computations.
- `useCallback()`: To memoize functions to prevent unnecessary ~~re-render~~ renders.

- `useState`: Add state to functional component.

Example:

```
const [count, setCount] = useState(0);
```

- `useEffect`: performs side effect (like fetching data or setting up intervals).

Example:

```
useEffect(() => {
  console.log("Component mounted or updated");
}, []);
```

- useContext: Accesses Context value

Example:

```
const theme = useContext(ThemeContext);
```

- useReducer: For complex state logic (like Redux Pattern).

Example:

```
const [state, dispatch] = useReducer(reducer, initialState);
```

10. Explain the process of creating React application along with Code snippets?

Ans: To create a new React project, follow these steps.

- Step 1: We need to install Node.js and npm (Node package manager)
Download link: <https://nodejs.org/en/download/prebuilt-installer>
 - Step 2: Verify Installation
 - Node: Go to the command prompt and type `node -v` or `node -version`
 - npm: Go to the command prompt and type `npm -v` or `npm -version`
 - Step 3: Create React App (CRA)
`npx create-react-app first-react-app.`
 - Step 4: Navigate to app directory
`cd first-react-app`
 - Step 5: Start the development server
`npm start`
- This command starts a development server and opens the default React app in your browser at `http://localhost:3000/`.